Professional Environments 2 - Student Expense Tracker

Module Code	CI5450
Module Title	Professional Environments 2
Assessment Title	Student Expense Tracker
Set by	Jad Abbass, Michal Bosy, Pushpa Kumarapeli and Tariq Rahim
Declare of use of GAI tools We have only used GenAI tools in this assignment for spelling and grammar check, making our writing more academic and to generate titles and organise my writing.	
The application can be accessed by following the link Expenses Tracker. If you are still experiencing issues, please contact Marco Silva	

Group 1		
Name	**KNumber**	**Participation**
Remi Eisaiankhongi	**k2421509**	**100%**
Marco Pereira Da Silva	**k2432045**	**100%**
Mahmood Jabary Sahbari	**k2453338**	**100%**
Simon Kowalski	**k1627462**	**100%**
Alicia Meikle	**k2326494**	**100%**
Amirali Hosseininezhad	**k2361414**	**100%**
Ali Yassine	**k2411096**	**100%**

1.0 Introduction

This report is to present the design, development, and evaluation of the Student Expense Tracker PowerApps solution that was made by Kingston University students for Kingston University students. This application was carefully designed and made to create a helpful platform that records expenses, manages monthly budgets, and helps students to better understand their spending habits.

The report documents the full development process, including analysis, design decisions, implementation across iterations, testing, and user guidance.

1.1 Problem Statement & Aim of the Project

Financial management is a common challenge for university students, who often need to balance limited income with multiple categories of spending.The aim of this project is to create a Student Expense Tracker PowerApps solution that supports Kingston University students in managing their finances.

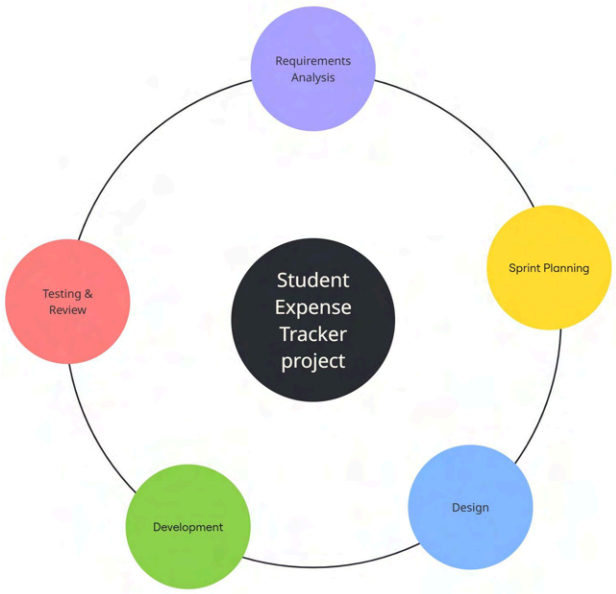
1.2 Project Overview Summary

Area	Key Information	
Target Users	Kingston University students	Undergraduate
	Students managing limited income	Students needing quick mobile access
User Needs	Track daily spending	Stay within monthly budgets
	Identify overspending	Improve financial awareness
	Make informed spending decisions	Build better financial habits
Platform Focus	Mobile-first design	Microsoft PowerApps solution
	Simple and accessible interface	Suitable for quick everyday use
Project Purpose	Help students manage personal finances	Provide clear spending records
	Support budget monitoring	Improve visibility of spending patterns

Area	Key Information	
Target Users	Kingston University students	Undergraduate
	Students managing limited income	Students needing quick mobile access
User Needs	Track daily spending	Stay within monthly budgets
	Identify overspending	Improve financial awareness
	Make informed spending decisions	Build better financial habits
Project Scope	Secure login	Expense recording
	Category selection	Expense list view
	Budget setting	Budget tracking
	Total spending calculations	Spending visualisation
Core Features	Authentication	Expense logging
	Categories	Expense list
	Monthly budgets	Visual budget tracking
Enhanced Features	Date filtering	Charts
	Edit/delete	Alerts

2.0 Development Approach

As a group, we took an agile and iterative approach to develop the Student Expense Tracker, and the picture below illustrates cycles of planning, design, development, and testing and this approach allowed us to deliver core features first and improve the application in later iterations.



2.3 Task Distribution and Weekly Progress

Task division was an important part of the team's workflow and all the work was allocated by knowing each member's role and strengths. We also did weekly progress reviews in order to monitor completed work, identify the next priorities, and keep development on schedule. These regular reviews acted as checkpoints for the team and helped ensure that everyone remained focused on the overall project goals.

3.0 Requirements Analysis

3.1 Requirements Gathering

The requirements for the Student Expense Tracker were identified through a structured review of the assignment brief, discussions within the project team, and consideration of the practical financial management needs of Kingston University students.

The assignment brief acted as the main source of requirements, as it clearly outlined the purpose of the application, the intended target users, and the must have, should have, and could have features expected in the final solution.

3.2 Functional Requirements

FR ID	Requirement	Reason
FR1	The system shall allow students to log in using university credentials	Secure authentication
FR2	The system shall allow users to add a new expense with amount, category, date and notes	Core expense tracking
FR3	The system shall display a list of logged expenses	Visibility of spending
FR4	The system shall allow filtering of expenses	Better review and search
FR5	The system shall allow users to set monthly budgets	Budget planning
FR6	The system shall calculate remaining budget	Financial awareness
FR7	The system shall display category totals	Spending analysis
FR8	The system shall allow editing and deleting expenses	Record maintenance
FR9	The system shall display charts of spending by category	Visual understanding
FR10	The system shall warn users when nearing/exceeding budget	Prevent overspending

3.3 Non-Functional Requirements

NFR ID	Requirement	Explanation
NFR1	Usability	The interface should be simple and mobile-friendly
NFR2	Performance	Screens should load quickly and calculations should update in real time
NFR3	Security	Access should use university authentication

NFR4	Reliability	Data should be stored consistently and accurately
NFR5	Maintainability	The solution should be easy to update in PowerApps
NFR6	Accessibility	Text, colours, and layout should be clear and readable

3.4 MoSCoW Prioritisation

Although the assignment brief already defined the MoSCoW priorities, the team also applied its own implementation-based prioritisation during development. This helped organise the order in which features were built, tested, and refined.

The team focused on bellow Development stages

Stage 1: Foundation

- Main screens and navigation
- Records screen
- Add Record form
- Data tables
- Early login/test access

Stage 2: Feature completion

- Filter options
- Date navigation arrows
- Budget functionality
- Visual analysis features
- Picture upload

Stage 3: Refinement

- Responsive layout across phone sizes
- Interface improvements
- Clearer analysis views
- General usability fixes

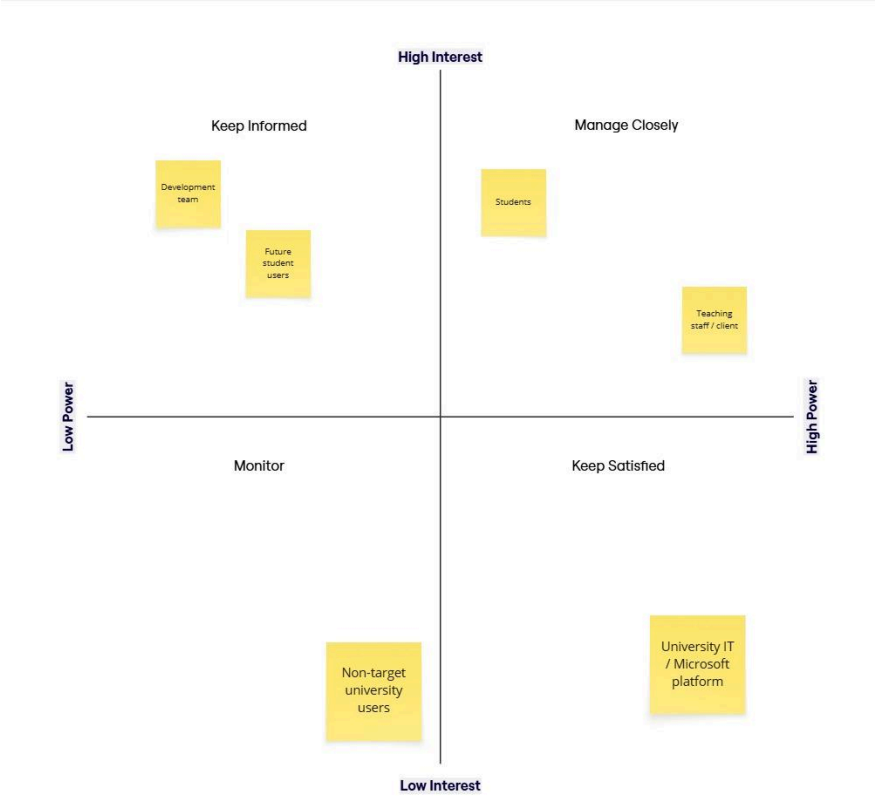
Stage 4: Lower-priority / future features

- Income tracking
- Recurring expenses
- Export to Excel
- Shared expenses
- Spending predictions

	Feature	Priority	Explanation
1	User authentication using university credentials	Must Have	Students must securely access the system using their university login to ensure privacy and correct user identification.
2	Add expense (amount, category, date, notes)	Must Have	Core functionality of the application. Without expense logging the system would not fulfil its purpose.
3	Expense categories (Food, Books, Transport, Entertainment, Housing, Other)	Must Have	Categories allow students to organise and analyse their spending behaviour.
4	Expense list view	Must Have	Students must be able to view all recorded expenses in order to manage and review their spending.
5	Budget setting (monthly or category budgets)	Must Have	Allows students to control their finances and plan spending effectively.
6	Budget tracking with colour indicators	Must Have	Visual indicators (green, yellow, red) help students quickly understand their spending status.
7	Basic spending calculations	Must Have	The system must calculate total spending, category totals, and remaining budget to provide meaningful insights.
8	Date range filtering (week/month/custom)	Should Have	Improves usability by allowing students to analyse spending within a specific time period.
9	Data visualisation (pie chart by category)	Should Have	Helps students easily understand spending patterns through graphical representation.
10	Edit and delete expenses	Should Have	Enables users to correct mistakes or update expense records.
11	Budget alerts when nearing limit	Should Have	Alerts warn students when they approach or exceed their budget limits.
12	Receipt photo capture	Should Have	Allows students to attach receipts for better financial tracking and record keeping.
13	Income tracking	Could Have	Allows users to track income and calculate net financial position, but not essential for the core system.
14	Recurring expenses	Could Have	Useful for subscriptions or regular payments but not required for initial functionality.
15	Additional charts (line or bar charts)	Could Have	Provides deeper insights but not necessary for basic system operation.
16	Export expense data to Excel	Could Have	Useful for advanced financial analysis outside the application.
17	Shared expenses with roommates	Could Have	Helpful for group spending situations but not critical for a personal expense tracker.
18	Spending insights and predictions	Could Have	Advanced analytical feature that could be added in future development.

3.5 Stakeholder Analysis

The stakeholder analysis was carried out to identify the main groups involved in the Student Expense Tracker project and to understand their level of power and interest. This helped the team decide which stakeholders needed the most attention during development. The findings showed that students and teaching staff were the most significant stakeholders because they had the greatest influence on the project and the strongest interest in its success. This analysis supported clearer decision-making and helped ensure that the system was designed around the needs of its most important users.



3.6 User Personas

The two personas shown below were created to represent typical Kingston University students who may use the Student Expense Tracker. They helped the team understand different user needs, behaviours, and financial challenges, and ensured that the design of the application

remained focused on real student experiences. By identifying goals, frustrations, preferences, and functional needs, the personas supported more informed design and development decisions throughout the project.

Daniel Mensah

Age: 21

Gender: Male

Location: Kingston upon Thames, London

Background

Daniel is a second-year Kingston University student who lives at home and commutes to campus several days a week. He also works part-time, which means he has both income and regular expenses to manage. His main spending areas include transport, food, phone bills, and course-related purchases. Because his routine is busy, Daniel does not want to spend a lot of time manually managing his finances, but he still wants a clear picture of how much he is spending each month.

Goals

- To keep track of spending while balancing study and part-time work
- To monitor transport and food expenses more closely
- To compare actual spending against planned budgets
- To spot patterns in monthly spending

Characteristics

- Busy and time-conscious
- Comfortable with technology
- Practical and goal-oriented
- Prefers fast and efficient tools
- Interested in visual summaries rather than detailed manual tracking

Technology Experience

- Regularly uses a smartphone and laptop
- Familiar with online banking and productivity apps
- Comfortable using digital tools for planning and organisation
- Prefers systems that save time and reduce effort

Challenges

- Limited time to manually organise finances
- Daily travel costs can add up quickly
- Hard to compare planned and actual spending without a clear system
- Can overlook patterns in spending over time
- Wants budgeting support without using a complicated finance tool

Preferences

- Prefers quick data entry and easy navigation
- Likes being able to filter expenses by date
- Wants charts to help visualise spending patterns
- Prefers a clean and professional interface
- Values alerts when spending approaches budget limits

Functional

- User should be able to filter expenses by week, month, or custom range
- User should be able to view spending by category through charts
- User should be able to edit or delete expense entries
- User should be able to receive alerts when nearing or exceeding budget limits
- User should be able to review total spending and category totals

Non-functional

- The system should be responsive on both mobile and desktop devices
- Data should update accurately when expenses are added or changed
- Visualisations should be clear and easy to interpret
- App should remain reliable during frequent use
- Navigation should be simple so users can complete tasks quickly

Aisha Rahman

Age: 18

Gender: Female

Location: Kingston upon Thames, London

Background

Aisha is a first-year Kingston University student who has recently moved away from home to study in London. She receives a student loan and occasional support from her family, so she has to manage her money carefully each month. Aisha often spends money on food, transport, study materials, and social activities, but finds it difficult to keep track of small daily purchases. She wants a simple way to understand where her money is going and avoid running out before the end of the month.

Functional

- User should be able to log in securely using university credentials
- User should be able to add expenses with amount, category, date, and notes
- User should be able to view all recorded expenses in a list
- User should be able to set monthly budgets for categories and overall spending
- User should be able to see remaining budget through visual indicators

Non-functional

- The app should be easy to use for students with little training
- The system should load quickly on mobile devices
- The interface should be clear, readable, and accessible
- The app should work reliably within the Microsoft PowerApps environment

Goals

- To record daily expenses quickly and easily
- To stay within a monthly budget
- To understand which categories take most of her money
- To avoid unnecessary overspending
- To build better financial habits while at university

Characteristics

- Organised but sometimes forgets to log small expenses
- Motivated to improve personal budgeting
- Prefers simple and clear interfaces
- Uses mobile apps regularly for everyday tasks
- Wants quick access to useful financial information

Technology Experience

- Uses a smartphone every day for communication, study, and banking
- Comfortable using mobile apps and university systems
- Prefers apps that are easy to navigate
- Expects fast and convenient access to information

Challenges

- Small daily purchases are easy to forget
- Budget can feel difficult to manage over a full month
- Hard to identify where overspending happens
- Limited financial experience as a new university student
- Can feel overwhelmed by too much financial information

Preferences

- Prefers a mobile-friendly design
- Likes clear categories such as food, transport, and entertainment
- Wants visual indicators to show remaining budget
- Prefers charts and summaries over long lists of numbers
- Values quick expense entry with minimal steps

3.7 User Stories

User story 1

Role: As a student user, I want to log in using my university account.
Goal: I want secure access to my personal expense records and budget information.
Reason: So my financial data is private and linked to my own account.
Acceptance Criteria: It is possible for the user to sign in using university credentials and access their own expense tracker dashboard.

User story 2

Role: As a student who wants to manage daily spending, I want to add expenses quickly.
Goal: I want to record the amount, category, date, and notes for each expense.
Reason: So I can keep track of where my money is going on a daily basis.
Acceptance Criteria: It is possible to add a new expense with amount, category, date, and optional notes, and the expense is saved correctly in the system.

User story 3

Role: As a student trying to organise spending, I want expenses to be grouped into categories.
Goal: I want to classify my spending into areas such as food, transport, books, entertainment, housing, and other.
Reason: So I can understand which areas of my life take most of my money.
Acceptance Criteria: It is possible to select a predefined category when recording an expense, and the expense is stored under that category.

User story 4

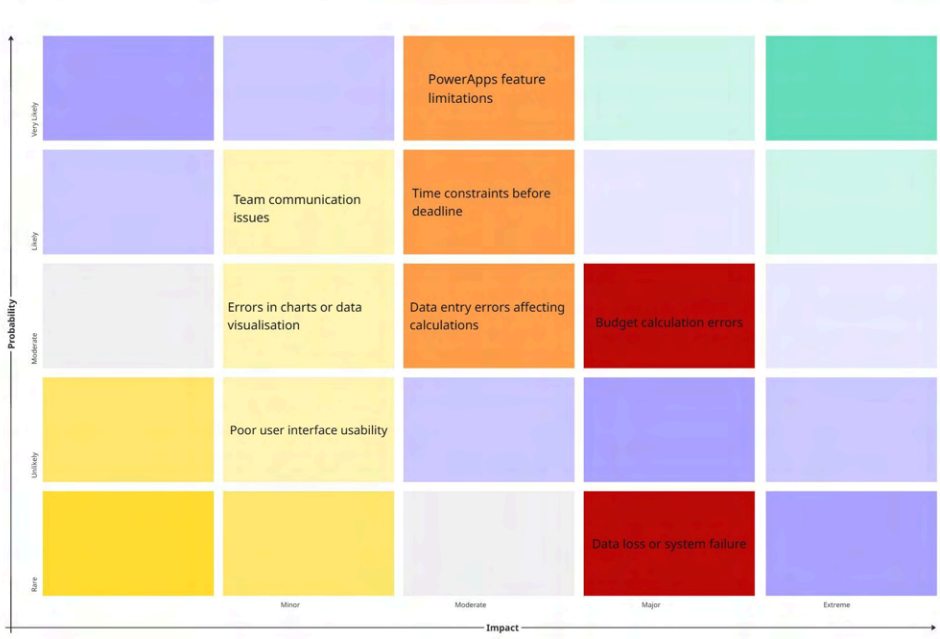
Role: As a student managing my finances, I want to view all my recorded expenses in one place.
Goal: I want to see a list of my expenses clearly and in order.
Reason: So I can review my spending history and check previous entries easily.
Acceptance Criteria: It is possible to display a list of all recorded expenses, showing key details such as amount, category, date, and notes.

User story 5

Role: As a student who wants to avoid overspending, I want to see my remaining budget.
Goal: I want the app to show how much of my budget is left using clear indicators.
Reason: So I can recognise when I am close to my limit and adjust my spending habits.
Acceptance Criteria: It is possible to view remaining budget amounts, and the system uses visual indicators such as green, yellow, and red to reflect budget status.

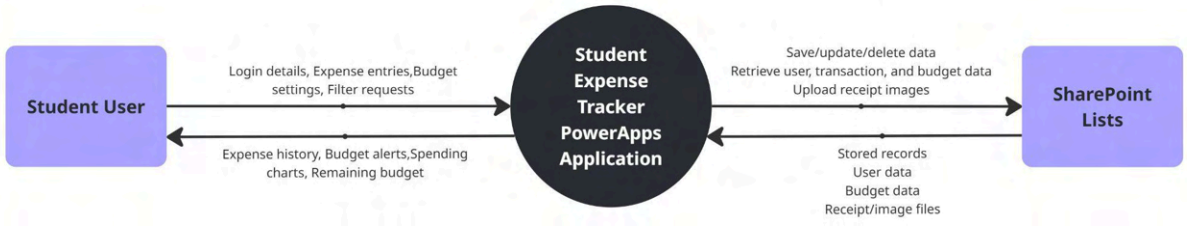
3.8 Risk Analysis

The risk analysis shown below was used to identify the main issues that could affect the success of the Student Expense Tracker project and to evaluate them in terms of likelihood and impact. This helped the team recognise which risks required the most attention during development, such as budget calculation errors, data loss, and PowerApps limitations. As a result, the team was able to plan more realistically, focus on the most serious risks, and take steps to reduce their effect on the project.



3.9 Context Diagram

This context diagram below illustrates the Student Expense Tracker a s the central system and shows the main external entities that interact with it.



4.0 Design

4.1 Design Goals

In this project we aimed to make the Student Expense Tracker a simple, mobile-first, visually clear system that Kingston students can use with minimal to no training and the interface was designed to support quick actions, reduce clutter, and present key information such as remaining budget and category totals in an immediately understandable format. In short, we have taken a minimalist approach to this application.

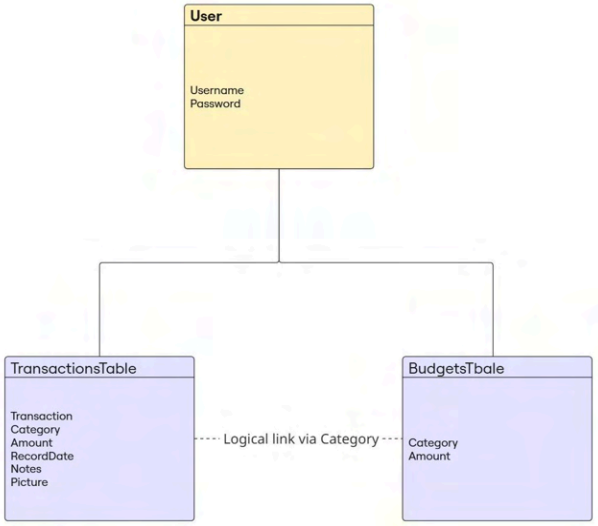
4.2 Use Case Diagram

The below use case diagram was created to show the main interactions between the student user and the Student Expense Tracker system. In this user case it identifies the key functions available within the application, including expense entry, expense management, budget setting, and spending review.

4.3 Class diagram

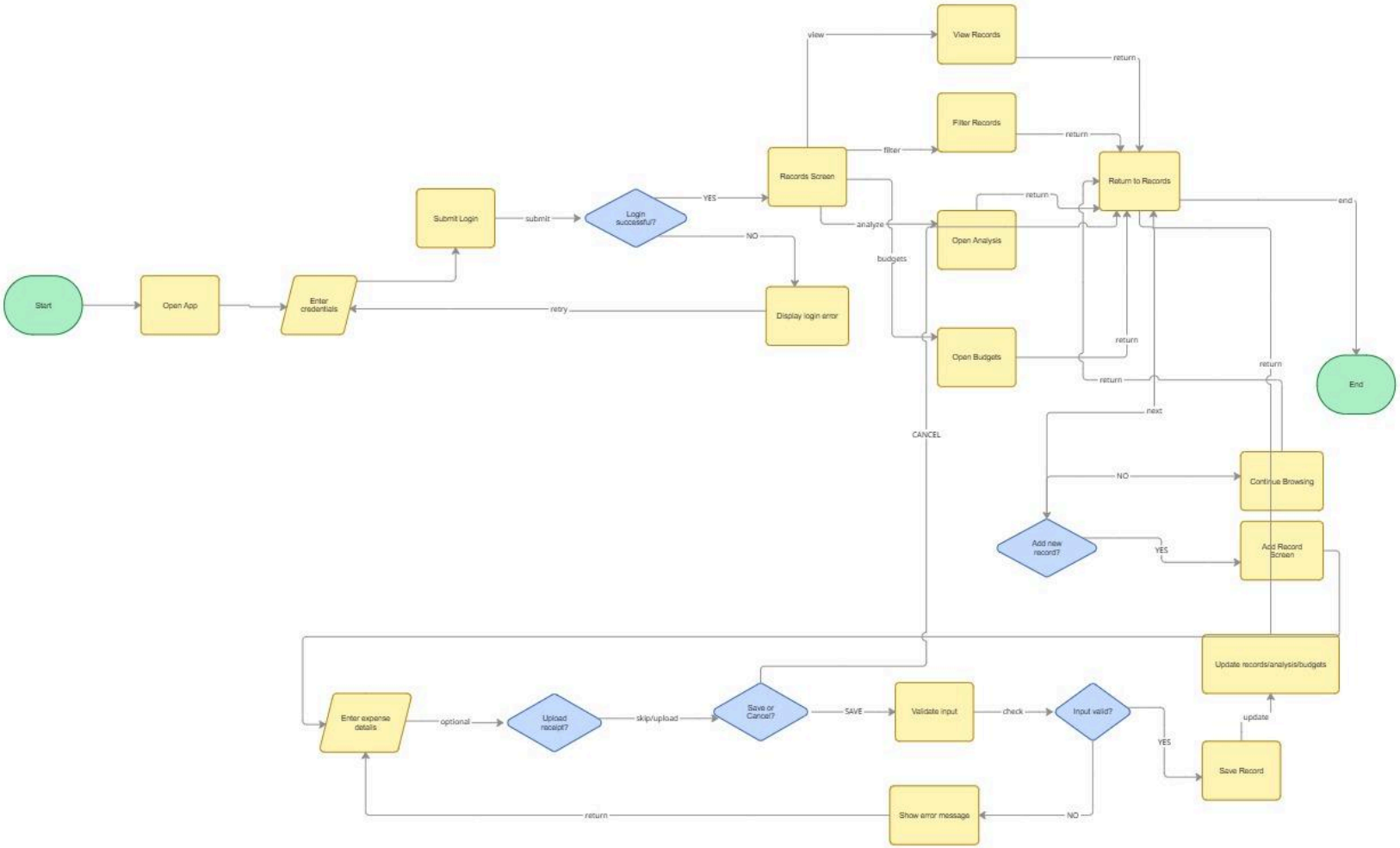
The diagram below shows the main SharePoint Lists used by the system, including **Users**, **TransactionsTable**, and **BudgetsTable** and the logical connection between transaction and budget data.

This structure demonstrates how login details, transaction records, and budget information are organised within the application to support the core functions of the Student Expense Tracker.



4.4 Activity Diagram for Key Processes

This activity diagram shows the flow of actions involved in the key processes within the Student Expense Tracker application.

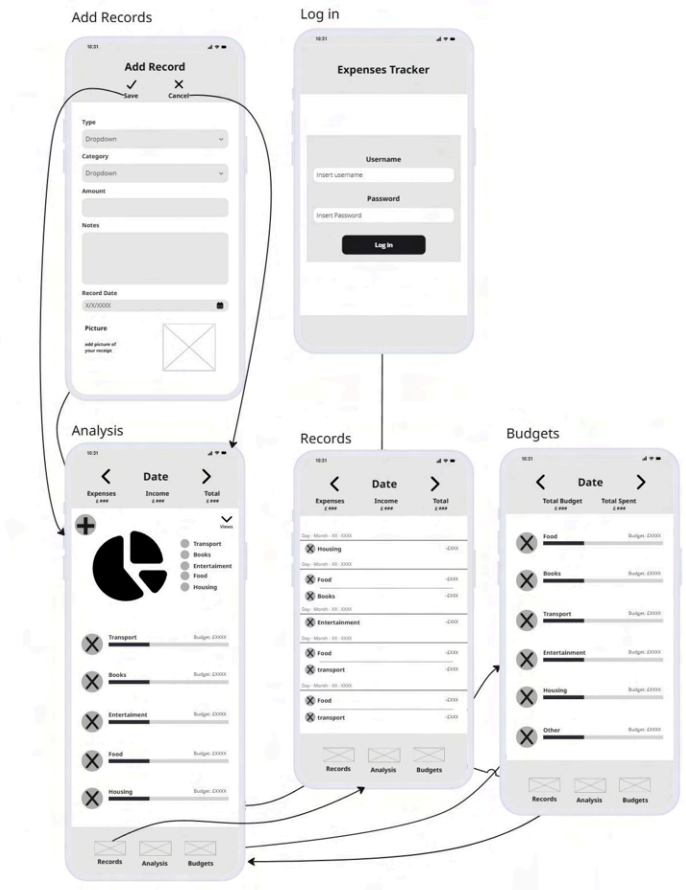

Start
Open App
Enter credentials
Submit Login
submit
Login successful?
YES
NO
retry
Display login error
Records Screen
view
View Records
return
filter
Filter Records
return
analyze
Open Analysis
return
budgets
Open Budgets
return
Return to Records
end
End
CANCEL
next
Add new record?
NO
Continue Browsing
YES
Add Record Screen
Enter expense details
optional
Upload receipt?
skip/upload
Save or Cancel?
SAVE
Validate input
check
Input valid?
YES
Save Record
update
Update records/analysis/budgets
NO
Show error message
return

4.5 Wireframes and Interface Design

The wireframe below was designed by the UI/UX designers and then shared with the development team as a visual guide for implementation.

The developers analysed the layout, navigation, and screen structure carefully and built the application to reflect the wireframe as closely as possible.

The design was intentionally kept simple, clear, and mobile-first to ensure that the application would be easy for students to use on their phones for everyday expense tracking.



4.6 Database / Data Schema Design

The Student Expense Tracker was designed using **Microsoft PowerApps** with **Microsoft Lists** as the main data source. The system uses three main tables: **Users**, **TransactionsTable**, and **BudgetsTable**. Each table was created to store a specific type of information required by the application.

Table	Purpose	Main Fields
Users	Stores user login details for accessing the application	Username, Password
TransactionsTable	Stores all financial records entered by the user, including income and expenses	Transaction, Category, Amount, RecordDate, Notes, Picture
BudgetsTable	Stores budget amounts for each spending category	Category, Amount

4.6.1 Relationships and Data Usage

The **TransactionsTable** and **BudgetsTable** are linked through the **Category** field, which allows the application to compare spending in each category against its corresponding budget.

4.7 Design Decisions and Trade-Offs

The design of the Student Expense Tracker was shaped by usability, platform constraints, and the needs of student users. Several design decisions were made to ensure that the application remained simple mobile-friendly and practical to implement in PowerApps and making these decisions lead up to trade-offs between functionality, clarity, and technical feasibility.

Design Decision	Option Chosen	Alternative Considered	Trade-Off / Limitation	Reason for Final Choice
Mobile-first layout	Simple mobile screen design	More detailed desktop-style layout	Less information can be shown on one screen	Students are more likely to use the app on their phones, so quick access and readability were prioritised
Simple interface	Clean and minimal screens	Feature-heavy screens with more data shown at once	Showing too much information would make screens crowded and harder to use	A simple interface improves usability and reduces confusion for student users
Separate screens for Records, Analysis, and Budgets	Information split across multiple screens	One combined dashboard screen	Users may need extra navigation between screens	Splitting content reduced clutter and made each screen easier to understand
PowerApps platform	Built in Microsoft PowerApps	Fully coded custom web or mobile application	PowerApps has design and functionality limitations compared with full custom development	PowerApps was required/appropriate for the project and allowed faster development within the Microsoft environment
Microsoft Lists as data source	Simple list-based storage	More advanced database solution	Less flexible than a full relational database	Microsoft Lists integrates well with PowerApps and was practical for the scope of the prototype
Predefined spending categories	Fixed categories such as Food, Books, and Transport	Fully custom user-created categories	Less flexibility for personal preference	Predefined categories made the app easier to use and supported clearer analysis and budget tracking

Optional receipt upload	Receipt image can be added when needed	Mandatory receipt upload for every transaction	Some records may not have visual proof attached	Making receipts optional keeps expense entry fast and avoids unnecessary effort for users
Basic visual analysis	Simple charts and category summaries	Advanced analytics and forecasting	Less detailed insight into long-term behaviour	Basic charts are easier to understand and more realistic within the scope of the project

5.0 Implementation

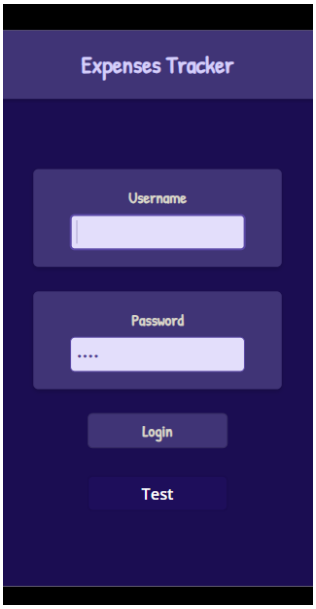
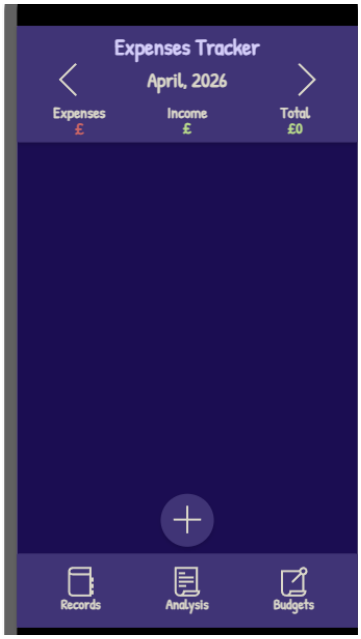
5.1 Development Environment and Tools

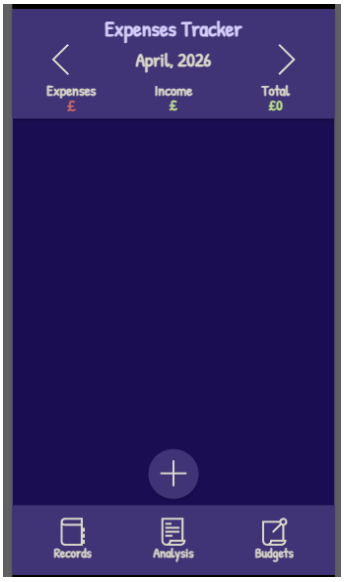
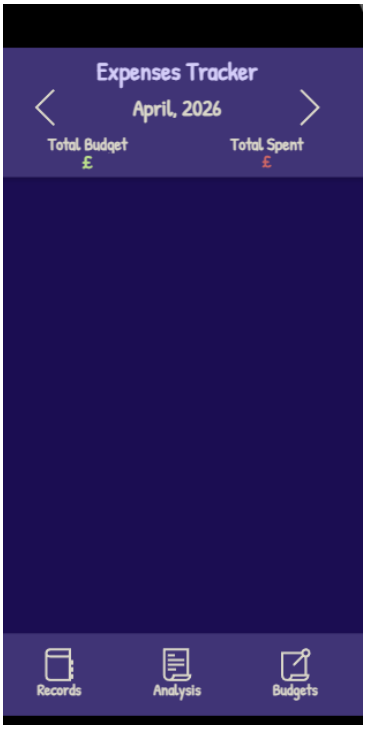
The Student Expense Tracker was developed using a small set of tools chosen to support design, implementation, organisation, and also for collaboration throughout all the stages of this project. The main development platform used was **Microsoft PowerApps.**

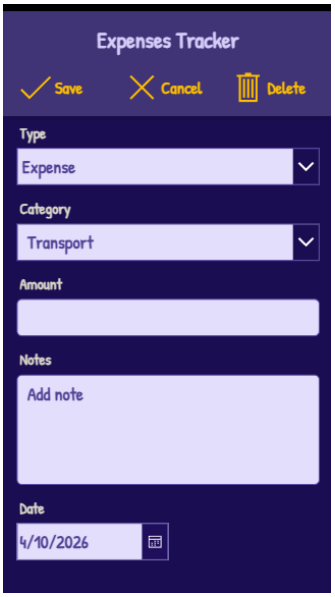
Tool / Platform	Purpose
Microsoft PowerApps	Main platform used to develop the application
Microsoft Lists	Data source used to store users, transactions, and budgets
Miro	Used to create wireframes and project diagrams
Microsoft Teams	Used for communication, file sharing, and collaboration
Calendar	Used to organise meetings, tasks, and deadlines

5.2 Iteration 1: Core Features

Iteration 1 focused on developing the core features required to create a working minimum version of the Student Expense Tracker and the screenshots below provide evidence of the main features completed during Iteration 1

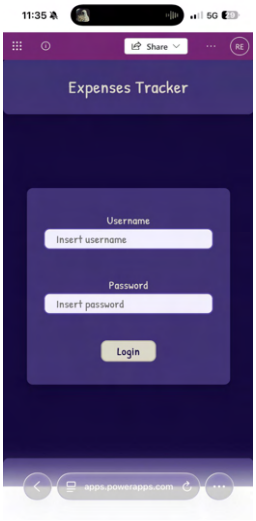
Screenshot	What it shows	Why it mattered in Iteration 1
Initial login screen with temporary test access button	Early login page with a bypass button for development	Allowed the team to continue building and testing the app before full authentication was completed
		
Initial Records screen layout before transaction data was added	Early structure of the Records page	Established the main screen users would use to review transactions
		

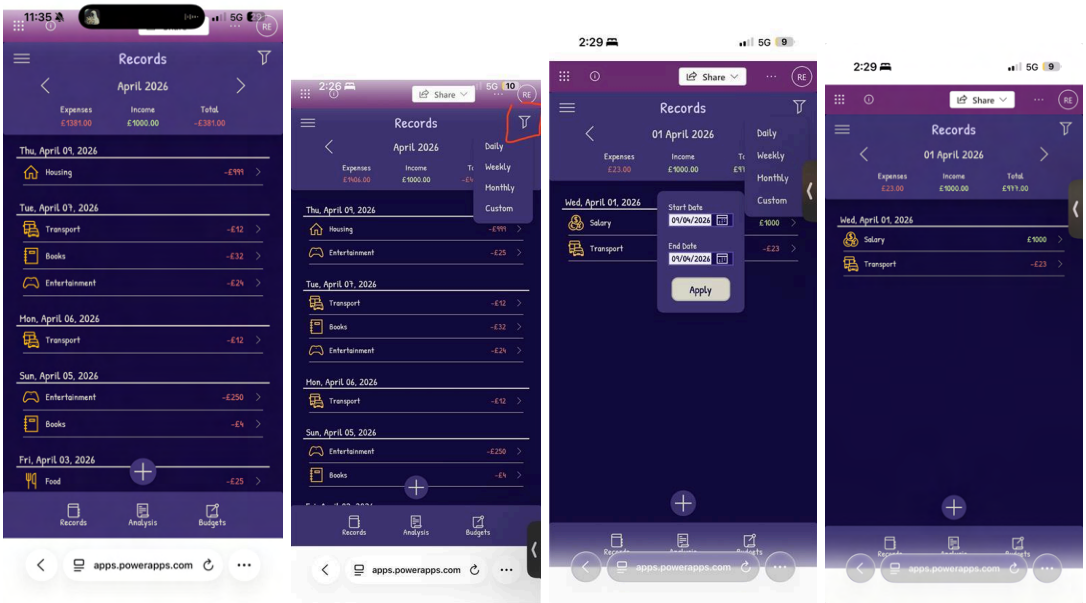
Initial Analysis screen structure before visual charts were implemented	First version of the Analysis page	Prepared the layout for later analysis and chart features
		
Initial Budgets screen / BudgetsTable structure	Early budget page and supporting data table	Created the basis for later budget tracking functionality
		

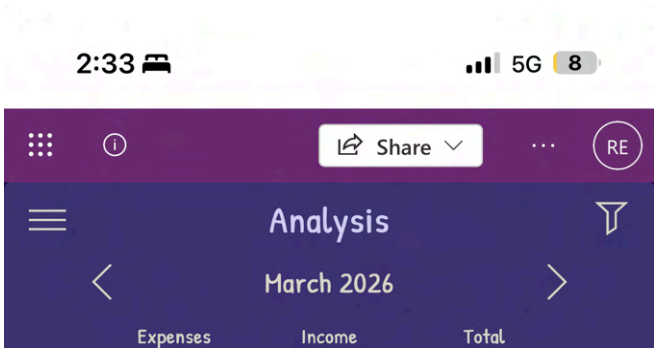
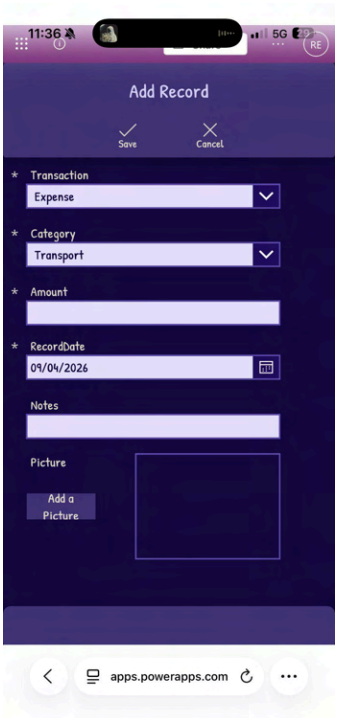
Early Add Record form used in Iteration 1 without picture field	First version of the input form	Enabled early transaction entry design before additional features were added
		

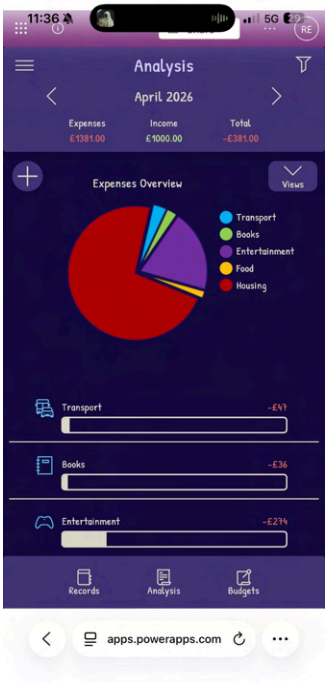
5.3 Iteration 2: Feature Completion, Usability Improvements and Refinement

Iteration 2 focused on improving the functionality, usability, and responsiveness of the Student Expense Tracker application.The screenshots below provide evidence of the main improvements and completed features delivered during Iteration 2.

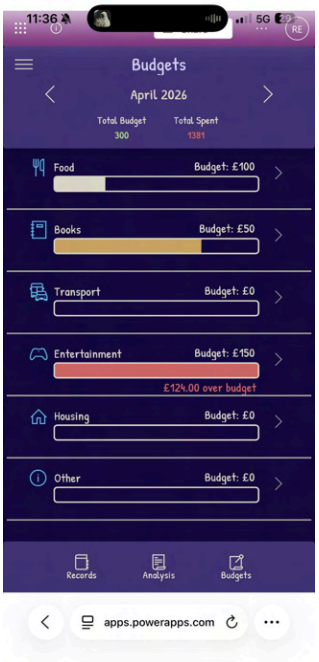
Working login screen / completed access flow	
	

Records screen with transaction data	Records screen with filter options
	

Working date navigation arrows	
 	
Completed Add Record form with picture upload	
	
Improved Analysis screen with pie chart	



Budget screen with working functionality



5.4 Key PowerApps Formulas, Implementation Challenges, and Solutions

5.4.1 Setting Filters

```
Set(selectedDateFilter, "Monthly");
Set(varShowFilterMenu, false);
Set(varShowDateFilterMenu, false);

Set(
    filterStartDate,
    If(
        selectedDateFilter = "Daily",
        selectedDate,
        selectedDateFilter = "Weekly",
        DateAdd(selectedDate, -Weekday(selectedDate, StartOfWeek.Monday) + 1, TimeUnit.Days),
        selectedDateFilter = "Monthly",
        Date(Year(selectedDate), Month(selectedDate), 1),
        selectedDateFilter = "Custom",
        customStartDate,
        selectedDate
    )
);

Set(
    filterEndDate,
    If(
        selectedDateFilter = "Daily",
        DateAdd(selectedDate, 1, TimeUnit.Days),
        selectedDateFilter = "Weekly",
        DateAdd(
            DateAdd(selectedDate, -Weekday(selectedDate, StartOfWeek.Monday) + 1, TimeUnit.Days),
            7,
            TimeUnit.Days
        ),
        selectedDateFilter = "Monthly",
        DateAdd(
            Date(Year(selectedDate), Month(selectedDate), 1),
            1,
            TimeUnit.Months
        ),
        selectedDateFilter = "Custom",
        DateAdd(customEndDate, 1, TimeUnit.Days),
        DateAdd(selectedDate, 1, TimeUnit.Days)
    )
);

ClearCollect(
    colFilteredTransactions,
    Filter(
        TransactionsTable,
        RecordDate >= filterStartDate &&
        RecordDate < filterEndDate
    )
)
```

This formula is used to filter transaction records by a selected date range. This allows the implementation of multiple periods such as daily, weekly, monthly, or custom range.

The “Set(selectedDateFilter, "Monthly");” is used to set a variable, so the app knows what type of filter the user applied.

The “Set(varShowFilterMenu, false);” and “Set(varShowDateFilterMenu, false);” is only used to close the menu after a filter selection has been made.

The Set for filterStartDate calculates the start day for the filter selected by the user. For Daily filter is simply the day selected, for weekly the formula calculates the Monday of that week, for Monthly calculates the first day of that month, and for custom is simply the customStartDate selected by the user.

The Set for filterEndDate calculates the end day for the filter selected by the user. For daily filter simply adds one to the selected date, weekly it adds 7 days to the Monday of the selected date, monthly it adds 1 Month to the first day of the selected date, and for custom it adds one day so the day selected can be included.

The ClearCollect creates a collection called colFilteredTransactions, to store only the records where RecordDate is between the filterStartDate and filterEndDate.

This formula was necessary to reduce code in many parts of the application, allowing us to build a table of records when a filter button is pressed, simplifying the code in the galleries, and when cycling through date periods with the arrow buttons.

One of the biggest challenges implementing this formula was making sure dates were being handled correctly for each filter, and ensuring dates were not being accidentally excluded.

Using global variables and collections in this case definitely improved readability of the code, and made it easier to manage across the whole application.

5.4.2 Budget Bar Colour

```
With(
    {
        categoryBudget: Value(ThisItem.Amount),
        categorySpent:
            Coalesce(
                Sum(
                    Filter(
                        colMonthlyExpenses,
                        Category = ThisItem.Category
                    ),
                    Amount
                ),
                0
            )
    },
    If(
        categoryBudget <= 0,
        RGBA(218, 215, 201, 1),
        categorySpent / categoryBudget < 0.5,
        RGBA(218, 215, 201, 1),
        categorySpent / categoryBudget < 0.8,
        RGBA(200, 164, 100, 1),
        RGBA(203, 102, 102, 1)
    )
)
```

This formula is used to determine the colour of the budgets bar based on how much was spent in each category compared to the budget set.

The With creates temporary variables, avoiding the creation of unnecessary global variables and variable cluttering.

With categoryBudget being the budget set for the category being calculated, the variable categorySpent is calculated by making use of a variable colMonthlyExpenses created before,

that filters expenses to be used in the budgets page by the filter applied to this page by default monthly, and filter those to match the category in question.

It then sums the Amount field of the result table to calculate the total spend for that category. The Coalesce ensures that if no value is found, the result will be 0 instead of blank.

Then the If sets the colour depending on the percentage between the categorySpent and categoryBudget.

The biggest challenge implementing this formula was ensuring that categories with no budget or no expenses did not cause calculation or computational errors. This was resolved with Coalesce and by checking whether the numbers were less than or equal to zero.

This formula was fundamental to implement the bar filling according to the percentage between spend and budget, providing the user with a more clear and visual interpretation of the budgets per category.

5.4.3 Gallery Transaction Days

```
SortByColumns(
    AddColumns(
        Distinct(
            Filter(
                AddColumns(
                    TransactionsTable,
                    OnlyDate,
                    DateValue(Text(RecordDate, "yyyy-mm-dd"))
                ),
                Switch(
                    selectedDateFilter,

                    "Daily",
                        OnlyDate = selectedDate,

                    "Weekly",
                        OnlyDate >= DateAdd(selectedDate, -Weekday(selectedDate, StartOfWeek.Monday) + 1) &&
                        OnlyDate <= DateAdd(selectedDate, -Weekday(selectedDate, StartOfWeek.Monday) + 7),

                    "Monthly",
                        Month(OnlyDate) = Month(selectedDate) &&
                        Year(OnlyDate) = Year(selectedDate),

                    "Custom",
                        OnlyDate >= customStartDate &&
                        OnlyDate <= customEndDate,

                    false
                )
            ),
            OnlyDate
        ),
        DisplayDate,
        Text(Value, "dd/mm/yyyy")
    ),
    "Value",
    SortOrder.Descending
)
```

This formula is used to generate a list of distinct transaction dates, so we can fill the first level gallery with the days, then to fill those days with the corresponding transactions, sorting the days in a descending order to show the most recent days first.

The AddColumns is first used to create a temporary column called OnlyDate, removing the time element from the RecordDate and keeping only the date part. This is important so we can treat Records with same date but different time, like they are the same, making the date a distinct value between this Records.

A Filter is applied to return only records that match the selected date range by the filters applied by the user, which is done by the Switch inside it.

The Distinct then ensures that dates only appear once, as this part of the gallery only requires days to appear once.

A second AddColumns is used to create another column called DisplayDate, formatting the date into a user friendly format.

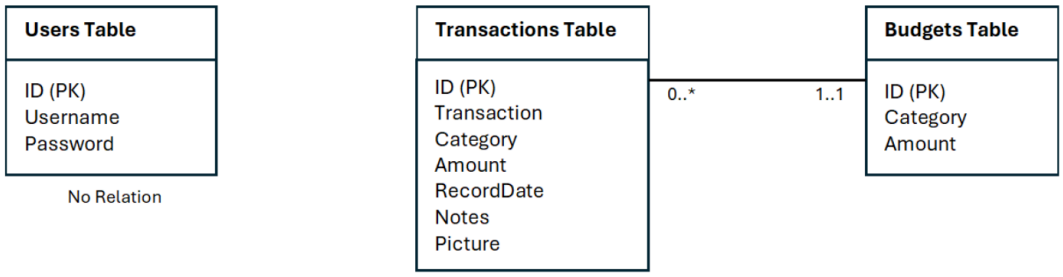
Finally, the SortByColumns sorts the values in descending order, displaying more recent day first.

This formula was necessary to group transactions by days instead of showing every transaction unstructured, creating a clean date structure that makes the interface easy to read and understand to the user.

The biggest challenge in this formula was handling the date values correctly, removing the time from DateTime, to ensure that days only appear once in the interface.

5.5 Database tables and structure

Tables Relation Used in the App



Data Dictionary Used in the App

Users Table		
ID	NUMBER	PRIMARY KEY
Username	VARCHAR(255)	NOT NULL
Password	VARCHAR(255)	NOT NULL

Transactions Table		
ID	NUMBER	PRIMARY KEY
Transaction	VARCHAR(255)	NOT NULL
Category	VARCHAR(255)	NOT NULL
Amount	CURRENCY	NOT NULL
RecordDate	DATE	NOT NULL
Notes	VARCHAR(255)	
Picture	IMAGE	

Budgets Table		
ID	NUMBER	PRIMARY KEY
Category	VARCHAR(255)	NOT NULL
Amount	CURRENCY	NOT NULL

Because of iterative development of the application, and the size of the project, we used the above relation for the tables.

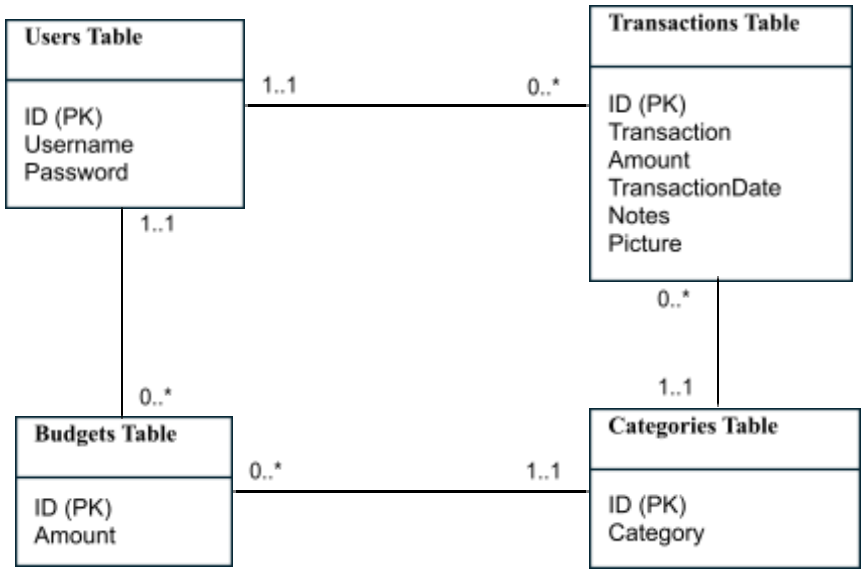
In this example, the transactions are only related to the budgets by the category's column with hard coded values, making each transaction indirectly have only one budget, and one budgets indirectly may have one or more transactions.

The use of hard coded values for categories is not right, and requires the use of huge formulas to get the values necessary for the display of components.

Because there is no relation between the users table and the rest, transactions and budgets will appear for every user connect to the application.
We are aware that this is not the correct implementation for a final delivery of the application, and correct implementation of the tables relations would require one more iteration of the development process.

Below is the correct relation between the tables, that would be implemented in a next iteration.

Correct Tables Relation



Entity Relation

A **user** may have one or more **transactions**, and may have one or more **budgets**.
A **budget** must have only one **user**, and only one **category**.
A **transaction** must have only one **user**, and only one **category**.
A **category** may have one or more **budgets**, and may have one or more **transactions**.

Correct Data Dictionary

Users Table		
ID	NUMBER	PRIMARY KEY
Username	VARCHAR(255)	NOT NULL
Password	VARCHAR(255)	NOT NULL

Categories Table		
ID	NUMBER	PRIMARY KEY
Category	VARCHAR(255)	NOT NULL

Budgets Table		
ID	NUMBER	PRIMARY KEY
UserID	NUMBER	NOT NULL, REFERENCES Users Table(ID)
CategoryID	NUMBER	NOT NULL, REFERENCES Categories Table(ID)
Amount	CURRENCY	NOT NULL

Transactions Table		
ID	NUMBER	PRIMARY KEY
UserID	NUMBER	NOT NULL, REFERENCES Users Table(ID)
CategoryID	NUMBER	NOT NULL, REFERENCES Categories Table(ID)
Transaction	VARCHAR(255)	NOT NULL
Amount	CURRENCY	NOT NULL
RecordDate	DATE	NOT NULL
Notes	VARCHAR(255)	
Picture	IMAGE	

With this table's relation, we would have each user connected to their own transactions and budgets.

The transactions and budgets would then be connected to categories to get the values necessary to assign them to the correct category.

This reduces the use of heavy formulas in Power Apps by being able to check CategoryID instead of having to build tables to display in Power Apps.

6.0 Testing

6.1 Introduction

This document contains the full testing artefacts for the Student Expense Tracker PowerApps project, produced by the testing team in line with the Test-Driven Development (TDD) approach required for this assignment. Test cases were defined early in the project lifecycle, before or during development of each feature and refined iteratively as the application evolved across two Agile sprints.

The purpose of this document is to demonstrate that the application has been systematically tested, that failures were identified and tracked, and that the final product meets the requirements specified in the brief.

6.2 Testing Strategy

Three levels of testing were carried out throughout the project:

6.2.1 Unit Testing

Unit testing was performed on individual features and form fields in isolation. This included verifying that input validation worked correctly (e.g. rejecting blank amounts), that each expense category saved accurately, and that budget calculations and colour indicators produced the correct outputs. Unit tests were written before each feature was built so that development could be guided by defined pass criteria.

6.2.2 Integration Testing

Integration testing checked that features worked together correctly as a connected system. This included verifying that a newly logged expense appeared in the records list, that deleting an expense updated the totals, and that the pie chart on the Analysis screen reflected live data from the SharePoint data source. Integration tests were introduced at the end of Sprint 1 and continued into Sprint 2.

6.2.3 User Acceptance Testing (UAT)

UAT was carried out with Kingston University student volunteers who had not been involved in building the application. They were given realistic tasks to complete without guidance such as logging an expense and checking their remaining budget and their experience was observed and recorded. UAT took place at the end of Sprint 2 once all core and enhanced features were in place.

6.3 TDD Approach Within Agile Sprints

The project followed an Agile methodology with two defined sprints. Testing was tied directly to this sprint structure at the start of each sprint, test cases were written for that iteration's planned features before development began. This meant the team knew exactly what pass criteria each feature needed to meet before anyone started building it in PowerApps.

For a no-code environment like PowerApps, TDD was adapted as follows:

- Sprint 1 test cases (TC001–TC017) written before core features were built covering authentication, expense logging, budget setting, and calculations

- Sprint 2 test cases written before enhanced features covering charts, filtering, edit/delete, receipt photos, and UAT
- Features built in PowerApps with those expected outcomes as the target
- Tests executed at the end of each sprint, results recorded
- Failures raised as bugs, fixes applied within the same sprint where possible, re-tested to confirm resolution

This sprint-by-sprint cycle define, build, test, fix, re-test reflects both the Agile iterative approach and the TDD principle of tests driving development rather than following it.

6.4 Test Case Table

The table below documents all test cases executed across the project. TC020 is marked FAIL and remains open as BUG003.

Test ID	Description	Input	Expected Outcome	Actual Outcome	Result	Comments	Date
TC001	Valid university login	Valid KU username + password	User authenticated, redirected to Records screen	User authenticated, redirected to Records screen	**PASS**		10/03/2026
TC002	Invalid credentials rejected	Wrong password entered	Error message displayed, user stays on login screen	Error message displayed: 'Invalid username or password'	**PASS**		10/03/2026
TC003	Blank credentials rejected	Username and password left blank	User cannot proceed, error shown	User cannot proceed, error shown	**PASS**		10/03/2026
TC004	Log a valid expense	Category: Transport, Amount: £100, Date: 10/04/2025, Notes: Car Insurance, Picture attached	Expense saved and appears in records list	Expense saved and appears in records list	**PASS**		12/03/2026
TC005	Submit expense with blank amount	Amount field left blank, all other fields complete	Validation error: 'Amount is required'	Validation error: 'Amount is required' shown in red	**PASS**		12/03/2026
TC006	Optional notes field left blank	Notes field empty, all	Expense saves without error	Expense saves without error	**PASS**		12/03/2026

Test ID	Description	Input	Expected Outcome	Actual Outcome	Result	Comments	Date
		other fields complete					
TC007	All 6 expense categories available	Category dropdown opened	Food, Books, Transport, Entertainment, Housing, Other all present	All 6 categories present and selectable	**PASS**		12/03/2026
TC008	Expense list displays all records	Multiple expenses logged across different dates	All expenses displayed grouped by date	All expenses displayed grouped by date	**PASS**		14/03/2026
TC009	Date range filter	Filter set to 01 Apr – 10 Apr 2026	Only expenses within that range shown	Only expenses within that range shown	**PASS**		14/03/2026
TC010	Edit an existing expense	Open existing Transport expense, change amount from £100 to £90	Updated amount saved and reflected in records	Updated amount saved and reflected in records	**PASS**		18/03/2026
TC011	Delete an expense	Open existing expense, press Delete	Expense removed from list, totals update	Expense removed from list, totals update	**PASS**		18/03/2026
TC012	Set per-category budget	Food budget: £110, Entertainment: £150, Books: £50	Budgets saved and displayed per category	Budgets saved and displayed per category	**PASS**		15/03/2026
TC013	Budget indicator — white (0–50% spent)	Spend under 50% of category budget	White/empty progress bar shown	White/empty progress bar shown	**PASS**		15/03/2026
TC014	Budget indicator — yellow (51–87% spent)	Spend between 51–87% of category budget	Yellow/gold progress bar shown	Yellow/gold progress bar shown	**PASS**		15/03/2026
TC015	Budget indicator — red (88%+ spent)	Spend at 88% or over of category budget	Red progress bar shown	Red progress bar shown	**PASS**		15/03/2026
TC016	Over budget alert message	Spending exceeds category budget	'X over budget' message shown in red below bar	'X over budget' message shown in red below bar	**PASS**	e.g. £149.00 over budget shown for Entertainment	15/03/2026

Test ID	Description	Input	Expected Outcome	Actual Outcome	Result	Comments	Date
TC017	Total spending calculation	Multiple expenses logged across categories	Total Spent figure updates correctly on budget screen	Total Spent figure updates correctly	**PASS**		14/03/2026
TC018	Pie chart renders on Analysis screen	Expenses logged across multiple categories	Pie chart visible showing proportional spending by category with legend	Pie chart visible with category legend	**PASS**		22/03/2026
TC019	Receipt photo — upload to expense form	Press 'Add a Picture', select image from device	Image preview shown on form	Image preview shown on form	**PASS**		25/03/2026
TC020	Receipt photo — saved after form submission	Expense with photo submitted via Save	Photo persists when expense is reopened	Photo not saved — field empty when record reopened	**FAIL**	Bug raised: BUG003 — currently open	25/03/2026
TC021	UAT — user logs expense end to end	Student user logs in, adds expense, views it in records	User completes task without guidance	User completed task successfully	**PASS**	Tested with student volunteer	02/04/2026
TC022	UAT — user sets budget and checks tracking	User sets category budgets and logs expenses	User can see progress bars and remaining amounts	User navigated budget screen successfully	**PASS**		02/04/2026
TC023	UAT — mobile usability	App used on mobile browser	All elements visible and tappable	All elements visible and tappable	**PASS**	Tested on both Android and iOS	02/04/2026

6.5 Bug Tracking and Resolution Log

All failures identified during testing are logged below. BUG001 and BUG002 were resolved within the project. BUG003 (receipt photo not saving) remains open at the point of submission.

Bug ID	Description	Severity	Discovered	Test Ref	Resolution	Resolved	Status
BUG001	Amount field accepted blank submission without validation error	**High**	12/03/2026	TC005	Added required field validation — 'Amount is required' error now shown in red	18/03/2026	**Resolved**

Bug ID	Description	Severity	Discovered	Test Ref	Resolution	Resolved	Status
BUG002	Budget indicator thresholds incorrect — yellow not triggering at right percentage	**Medium**	15/03/2026	TC014	Corrected PowerApps If statement thresholds: white 0–50%, yellow 51–87%, red 88%+	24/03/2026	**Resolved**
BUG003	Receipt photo uploads successfully but does not persist after saving the expense record	**Medium**	25/03/2026	TC020	Under investigation — believed to be a SharePoint column data type issue	—	**Open**

6.6 Test Evidence Screenshots

The following screenshots provide visual evidence of the application being tested. Each is labelled with the corresponding test case ID

TC001 — Successful Login

Expected: User authenticated and redirected to the Records screen. Result: PASS

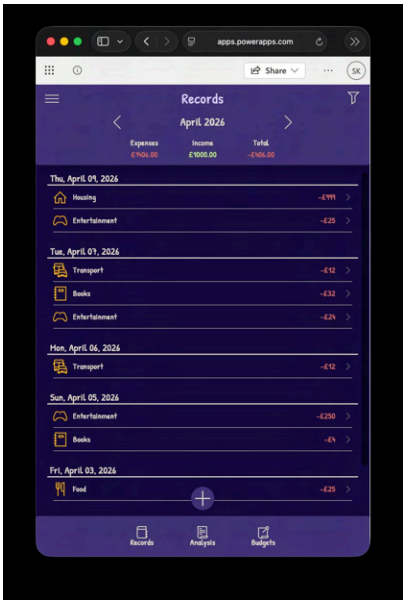


Figure 1: Records screen displayed after successful login (TC001)

TC002 — Invalid Credentials Rejected

Expected: Error message shown, user stays on login screen. Result: PASS

Figure 2: 'Invalid username or password' error displayed (TC002)

TC005 — Blank Amount Validation

Expected: 'Amount is required' error shown when amount field left blank. Result: PASS

Figure 3: Validation error shown for blank amount field (TC005)

TC004 — Valid Expense Logged with Receipt Photo

Expected: Expense form completed with all fields including photo. Result: PASS

Figure 4: Completed expense form with receipt photo attached (TC004)

TC008 — Expense Appears in Records List

Expected: Newly saved expense visible in records list. Result: PASS

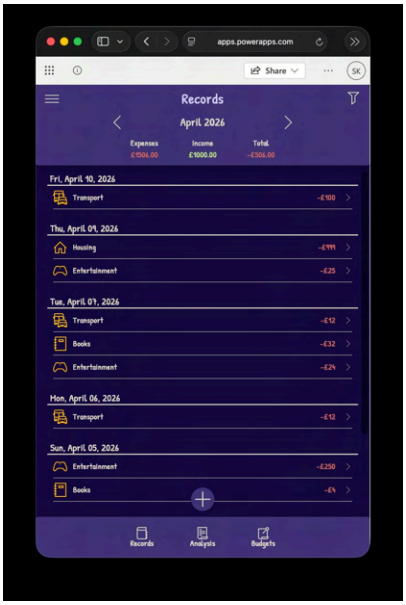


Figure 5: Records list showing saved expenses grouped by date (TC008)

TC009 — Date Range Filter

Expected: Only expenses within selected date range displayed. Result: PASS

Figure 6: Records filtered to 01 Apr – 10 Apr 2026 (TC009)

TC010 — Edit Existing Expense

Expected: Expense can be opened and modified. Result: PASS

Figure 7: Edit Record screen with amount changed (TC010)

TC011 — Delete Expense

Expected: Expense removed from list after deletion. Result: PASS

Figure 8: Records list updated after expense deleted (TC011)

TC013 / TC014 / TC015 — Budget Colour Indicators

Expected: White bar under 50%, yellow 51–87%, red 88%+. Result: PASS

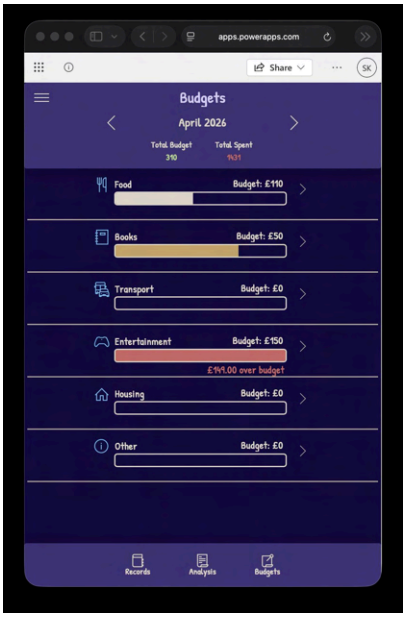


Figure 9: Budget screen showing white (Food), yellow (Books) and red (Entertainment) indicators (TC013/014/015)

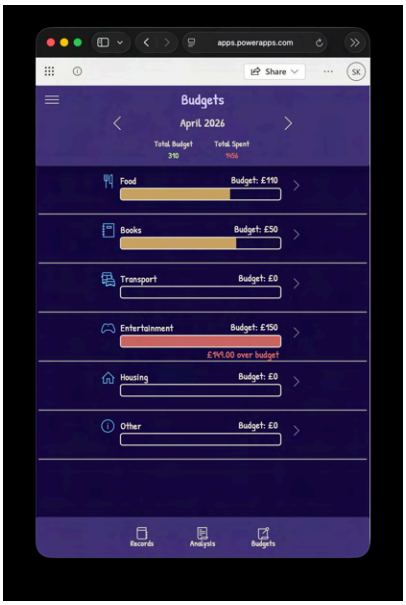


Figure 10: Yellow indicator visible on Books category at ~72% spent (TC014)

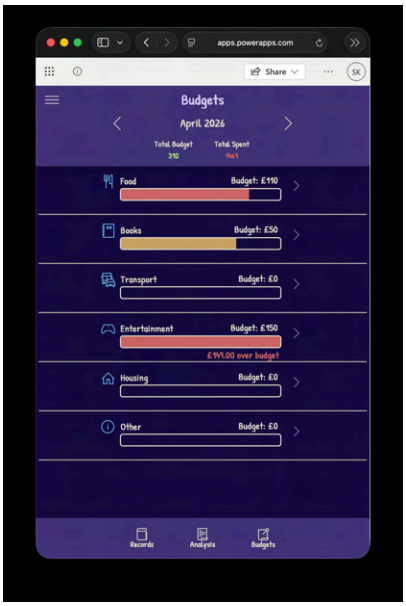


Figure 11: Red indicator on Food category at 88%+ spent (TC015)

TC016 — Over Budget Alert

Expected: 'X over budget' message shown when category budget exceeded. Result: PASS

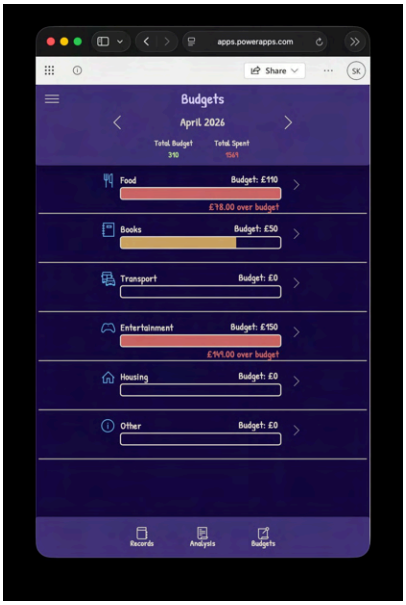


Figure 14: Over budget alert shown — '£78.00 over budget' for Food (TC016)

TC017 — Total Spending Calculation

Expected: Total Spent figure updates correctly on budget screen. Result: PASS

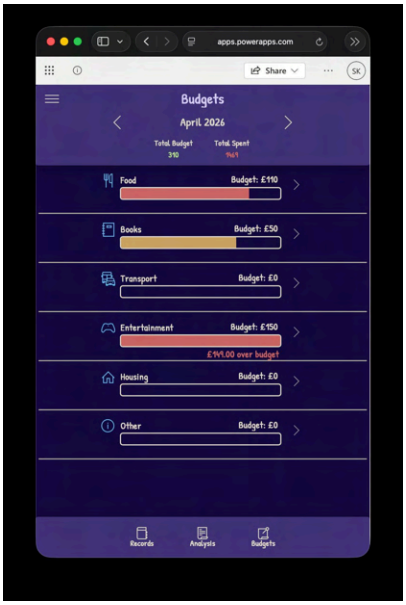


Figure 13: Budget screen showing Total Budget £310 and Total Spent £1469 (TC017)

TC018 — Pie Chart on Analysis Screen

Expected: Pie chart showing proportional spending by category with legend. Result: PASS

Figure 12: Analysis screen showing expenses overview pie chart by category (TC018)

TC019 / TC020 — Receipt Photo (BUG003)

TC019 PASS: Photo uploads and previews correctly on the Add Record form. TC020 FAIL: Photo does not persist after saving — field is empty when record is reopened. Bug logged as BUG003 (open).

Figure 15: Receipt photo displayed on form during entry (TC019 PASS) — photo does not save on submission (TC020 FAIL / BUG003)

6.7 Test Summary

A total of 23 test cases were executed across unit, integration, and user acceptance testing. Of these, 22 passed and 1 remains a known open defect (BUG003 — receipt photo not persisting). Two earlier bugs (BUG001 and BUG002) were identified, fixed, and re-tested successfully within the sprint they were found.

The receipt photo upload feature (TC019) works correctly, the image displays on the form. The defect is specifically in the saving of the image to the SharePoint data source, which is believed to be a column data type configuration issue. This has been logged and is under investigation.

Expenses Tracker – User Manual

System Overview

Expenses Tracker is an application designed to help students track their finances.
Users can log their expenses and income, set budgets, and visualise spending in real time, giving users with full control over their money.

Getting Started

1. Accessing Expenses Tracker App

The application can be accessed by following the link Expenses Tracker.
The app can be used in the web version, or it can be downloaded to the Power Apps platform by pressing the download button. *(Figure 1)*

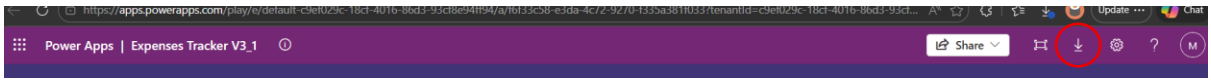


Figure 1

(Professors can use the link Expenses Tracker Power Apps to access the editing part. Because Power Apps requires to give access, I added every professor in the assignment brief to the Power Apps and Share Point Lists. If you are still experiencing issues, please contact Marco Silva)

2. Login

2.1 Open the application

2.2 Enter username and password *(Figure 2)*

(Available logins according to assignment brief:

Username	Password
K2432045	pass
K2421509	pass
K2361414	pass
K2453338	pass
K1627462	pass
K2326494	pass
K2411096	pass

)

Figure 2

2.3 Press the “Login” button

After logging in, you will be redirected to the Records page.

Interface Overview

1. Records Page

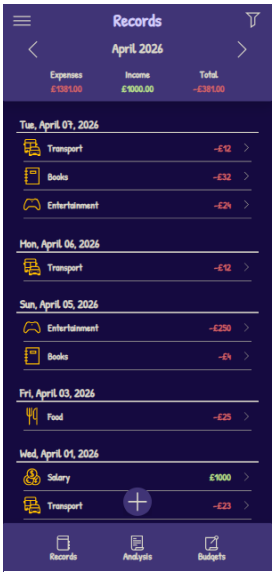


Figure 3

1.1 The first page the user encounters after logging in.

1.2 This page shows all the expenses and income added by the user to the application. *(Figure 3)*

1.3 The add button near the bottom of the page, allows the user to add an expense or income. *(Figure 4)*


Figure 4

2. Header Menu

2.1 The header menu displays information about the current page, current date period selected, and total expenses, income, and total amount for the date period selected by the user. *(Figure 5)*

Figure 5

2.2 A filter button on the header menu, allows the user to set different filter ranges. *(Figure 6)*

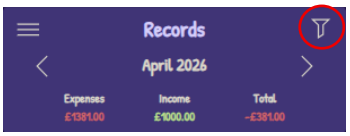


Figure 6

2.3 Navigate between date ranges selected by the filter, with the left and right arrows located in the header menu. *(Figure 7)*

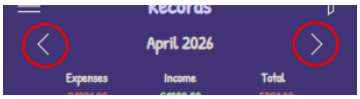


Figure 7

3. Navigation Menu

Allows the user to switch between records, analysis, and budgets views. *(Figure 8)*

Figure 8

4. Manage Records Page

Used to create, update, or delete records.

Users can input transaction details such as type, category, amount, date, notes, or pictures. *(Figure 9)*

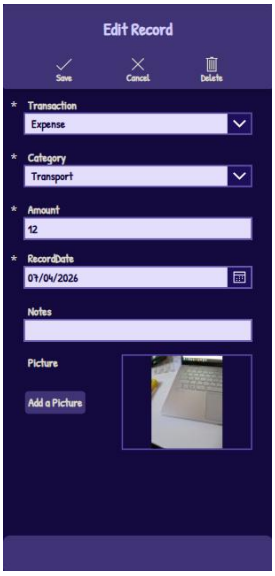


Figure 9

5. Analysis Page

Provides a visual representation of financial data using pie charts and line graphs. *(Figure 10 and 11)*

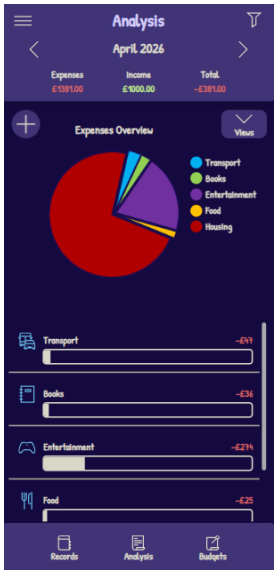


Figure 10

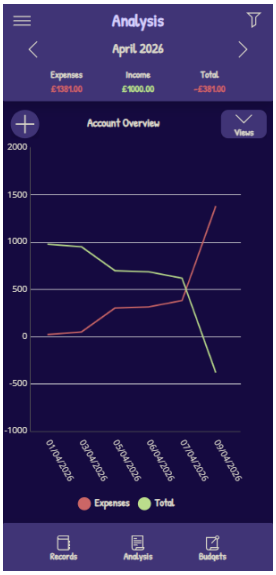


Figure 11

6. Budgets Page

Allows users to set and track monthly budgets for different categories. *(Figure 12)*

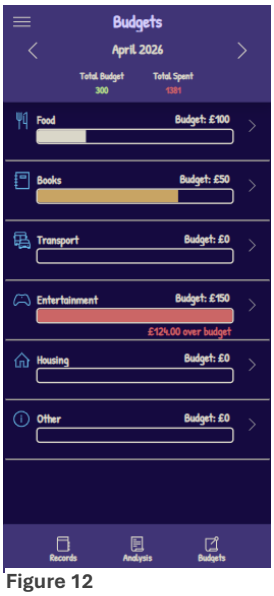


Figure 12

Features

1. Managing Records

1.1 Add a Record

Navigate to Records page.

Press the Add button.

(Figure 13)

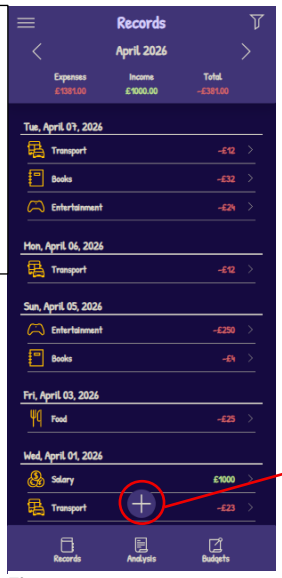


Figure 13

Enter transaction details.

Press “Save” to confirm or “Cancel” to discard.

(Figure 14)

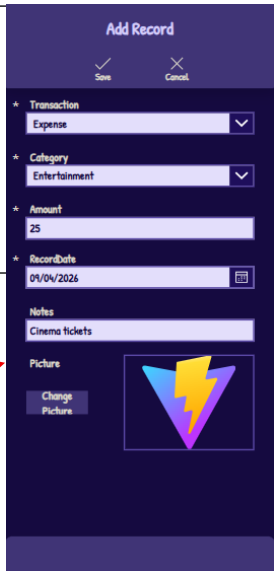


Figure 14

1.2 Edit or Delete an existing Record

Click on a record from the Records page.

(Figure 15)

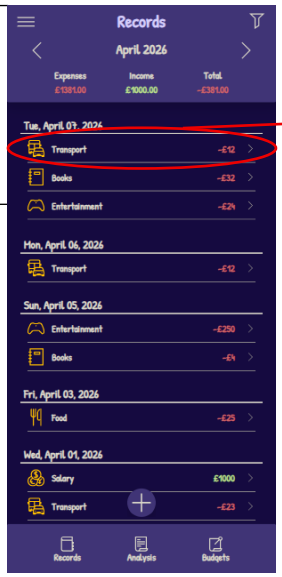


Figure 15

Modify the required fields.

Press “Save” to update, “Cancel” to discard, or “Delete” to remove the record.

(Figure 16)

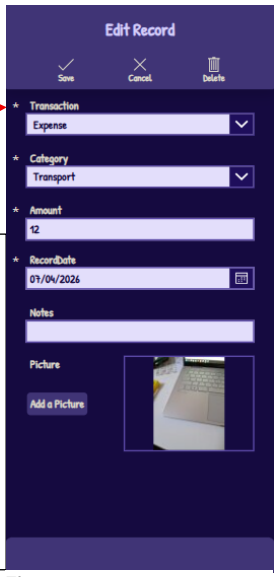


Figure 16

2. Analytics Views

2.1 Category expenses overview

Display spending distribution across categories using a pie chart.

If the view is not set, press the “Views” button.

(Figure 17)

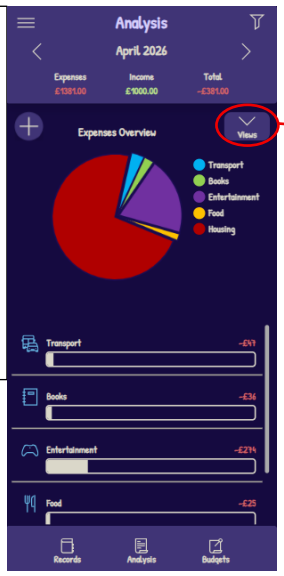


Figure 17

Select “Expenses Overview” to set the view.

(Figure 18)

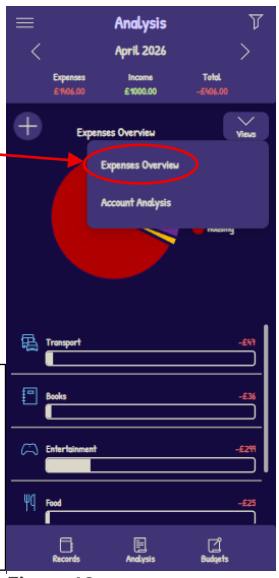


Figure 18

2.2 Account analysis overview

Click on “Views” button.

Select “Account Analysis” to set the view.

(Figure 19)

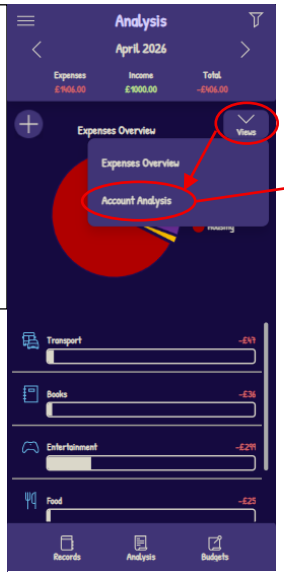


Figure 19

Displays income and spending trends over time using a line chart.

(Figure 20)

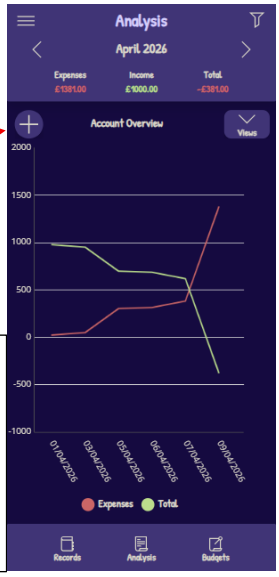


Figure 20

3. Managing Budgets

3.1 Set and Edit budgets

Click on a category.

(Figure 21)

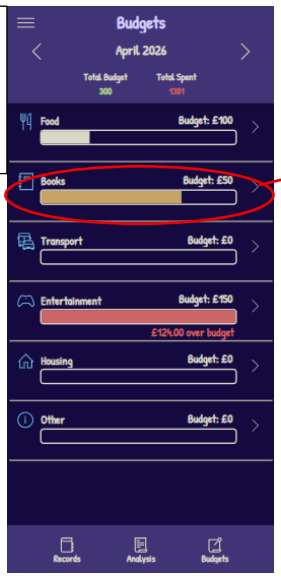


Figure 21

Insert budget amount for the chosen category.

Press “Save” to update or “Cancel” to discard.

(Figure 22)

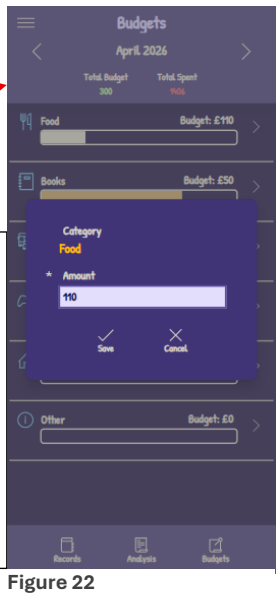


Figure 22

3.2 Viewing budgets

Viewing budgets in Expenses Tracker is easy and intuitive.

The header menu displays the total amount of all budgets, and the total spend between all categories.

If a budget is set to 0, that will not count towards total budget, and will not display any warnings of limits for those categories.

If a budget is set, a bar will fill up providing easy visualisation of the stage, and there are 3 stages for the budget:

- 0% to 49% - Normal
- 50% to 79% - Warning (Yellow)
- 80% to 100% - Critical (Red)

When spending for that category exceeds 100% of the budget amount, a notification will appear under the bar, notifying the user the amount used over the budget.

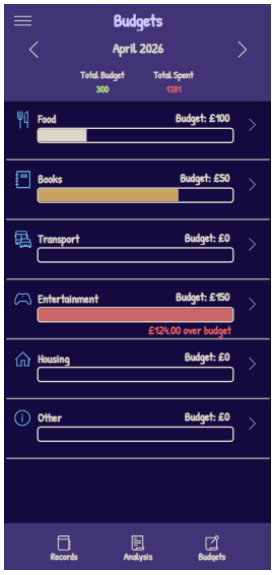


Figure 23

4. Managing Filters

4.1 Applying daily, weekly and monthly filters

Applying filters in Expenses Tracker can be done by pressing the filter button on the top right corner of the header menu, as shown in Figure 24.

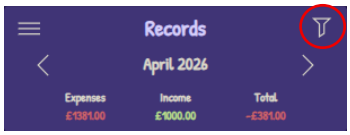


Figure 24

Filters are available on the records and analysis pages, as the budgets page uses monthly budgets.

Pressing the filter button will open a drop-down menu that allows the user to set daily, weekly, monthly, or custom filters to the records displayed, as shown in Figure 25.

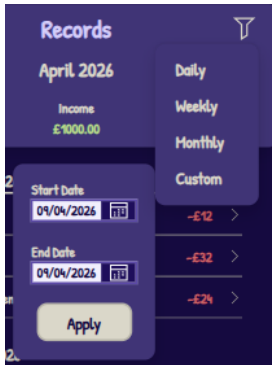


Figure 25

Page date and records change according to the filter chosen, as shown in Figures 26, 27, 28, and 29.

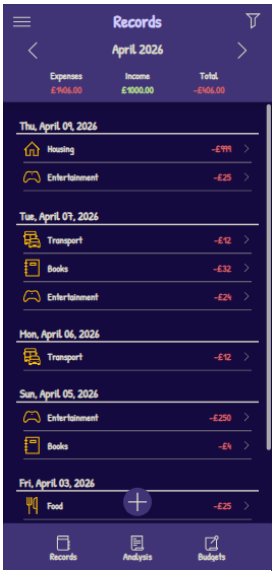


Figure 26

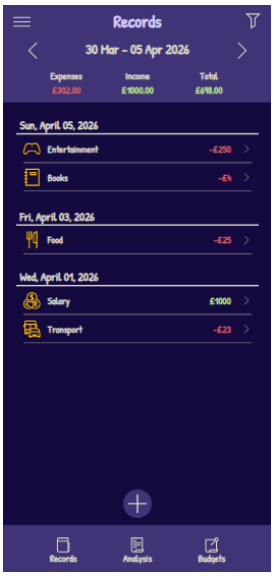


Figure 27

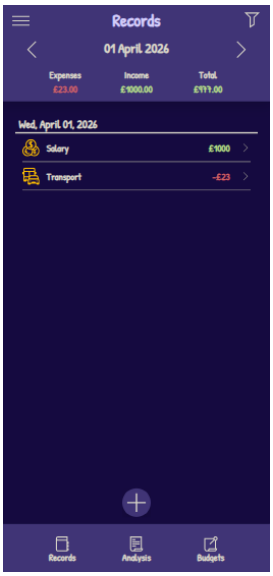


Figure 28

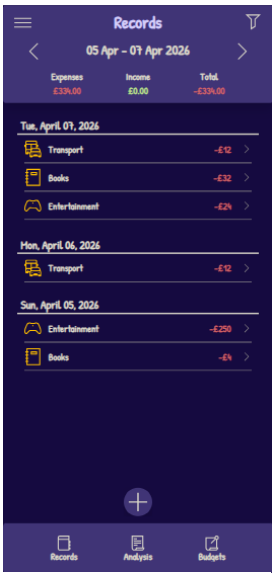


Figure 29

Filter ranges can be cycled through using the header menu left and right arrows, shown in Figure 30.

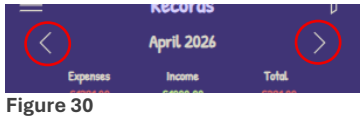


Figure 30

Troubleshooting

1. Cannot log in?
Ensure that the username and password are entered correctly, including upper-case and lower-case characters.
If the issue persists, try restarting the application and logging in again.

2. Data not showing?
Ensure that a filter is not limiting the displayed data, by changing the selected date range.
If still having issues, try refreshing the application and logging in again.

3. Slow or unresponsive application?
Close and reopen the application.
Ensure you have a stable internet connection, as the app relies on online services.

FAQ

1. Can I add my income?
Yes, on the records or analysis page, click the add button, select transaction type “Income”, and just fill in the rest of the fields.

2. Can I set different categories for my income?
Yes, at the moment we have 3 categories, salary, sale, and other. Just select it from the manage records screen when adding or editing a record.

3. Can I use the app on a computer?
Yes, the app was optimised to be used on mobile devices, but it still works on any computer.
Using the application on a computer will not allow users to take pictures when adding or editing records.

4. Can I see my spending per category?
Yes, just open the analysis page, and make sure you are in the “Expenses Overview” page.
In there you are able to visually see the spending per category, and how much that category percentage spending is compared to the total.

5. What happens if I exceed my budget?
A notification will display in the budgets page, under the exceeded category.
The notification will display in red, and will show the total amount exceeded.

Appendix

1.1 Team Roles

The project team consisted of seven members, the roles were carefully distributed to make sure that all key areas of the project were covered effectively. As the team was required to take on a range of responsibilities, some tasks were shared between two members, and some individuals took on more than one role where appropriate.

1.1.1 Team Member Responsibilities

Remi EisaianKhongi
Project Manager & Lead Developer
• Coordinated team progress and deadlines
• Organised tasks and monitored milestones
• Supported backend-related development
• Helped manage data structure and system logic

Mahmood Jabary Sahbari
UI/UX Designer
• Designed the user interface and user experience of the application
• Contributed to screen layouts and visual consistency
• Helped improve usability for student users
• Supported the creation of a clear and accessible design

Marco Pereira Da Silva
Backend Developer
• Led the overall development of the application
• Implemented key features in PowerApps
• Supported integration of system components
• Ensured the core functionality of the app worked correctly

Alicia Meikle
UI/UX Designer
• Contributed to interface design and layout planning
• Supported the creation of user-friendly screens
• Helped improve accessibility and clarity
• Assisted in strengthening overall user interaction

Amirali Hossininezhad
Developer & Documentation

• Supported application development and feature implementation
• Assisted with technical problem-solving
• Contributed to documenting the project process
• Helped record system features and technical details

Simon Kowalski
Lead Tester & Communications Lead
• Led testing activities across the project
• Created and reviewed test cases
• Recorded bugs and testing outcomes
• Managed team communication and helped keep members aligned

Ali Yassine
Lead Analyst
• Led the analysis stage of the project
• Gathered and organised requirements
• Supported stakeholder and user analysis
• Helped define project scope, priorities, and system need